Tree Algorithms Mastery: LCA, Tree Diameter, Euler Tour & Centroid Decomposition

Curated Problemset & Competitive Programming Reference | Tailored for ICPC Dhaka & National IUPCs

1. Overview & Learning Progression

Trees represent the single most heavily tested graph category in Bangladeshi competitive programming (accounting for 52 of 343 problems in 2021–2026). Mastering these four techniques builds the core foundation required to solve mid-to-late contest problems:

- **1. Tree Diameter:** Longest path computation via 2-DFS ($O(N)$) and Tree DP.
- **2. Binary Lifting & LCA:** Ancestor queries, path distances, and path bottleneck queries in $O(\log N)$.
- **3. Euler Tour Technique:** Flattening trees to 1D ranges for subtree and root-to-node path queries.
- **4. Centroid Decomposition:** Divide and conquer on trees, solving path counting problems in $O(N \log N)$.

2. Pillar 1: Tree Diameter & Longest Paths

A. Theoretical Foundation

- **Two-DFS Method (O(N)):** Run DFS from arbitrary node 1 to find farthest node U. Run second DFS from U to find farthest node V. Distance dist(U, V) is the diameter. Only valid for non-negative edge weights.
- **Tree DP Method (O(N)):** Let down[u] be the longest downward path from u to a leaf. The longest path turning at u is max(down[v1] + down[v2] + w1 + w2) over its two largest child branches. Valid for negative edge weights.

B. Two-DFS Implementation Template (C++20)

```
const int MAXN = 200005;
vector<pair<int, long long>> adj[MAXN];
long long dist_node[MAXN];

void dfs(int u, int p, long long d, int &farthest_node) {
    dist_node[u] = d;
    if (dist_node[u] > dist_node[farthest_node]) farthest_node = u;
    for (auto &[v, w] : adj[u]) {
        if (v != p) dfs(v, u, d + w, farthest_node);
    }
}
pair<long long, pair<int, int>> get_diameter(int n) {
    int u = 1; dfs(1, 0, 0, u);
    int v = u; dfs(u, 0, 0, v);
    return {dist_node[v], {u, v}};
}
```

C. Curated Practice Ladder

Problem & Platform	Tier	Core Concept / Key Intuition
CSES 1131 - Tree Diameter	Core	Standard benchmark: Implement both 2-DFS and Tree DP methods.
SPOJ PT07Z - Longest path	Core	Classic 0/1 unweighted tree diameter

		verification.
CF 1404B - Tree Tag	Med	Tree diameter + Game Theory: If jump range >= ceil(diameter/2), evasion is impossible.
CF 455C - Civilization	Med-Hard	Maintaining diameters dynamically while merging trees via DSU.
ICPC Dhaka Reg 2024 - F. Flowers	Hard	Forest diameters and optimal bridge additions.

3. Pillar 2: Lowest Common Ancestor (LCA) via Binary Lifting

A. Theoretical Foundation

- **Precomputation Table:** up[u][k] stores the 2^k-th ancestor of node u. up[u][k] = up[up[u][k-1]][k-1]. Precomputation takes O(N log N) time and memory.
- **LCA Query in O(log N):** Equalize depths of deeper node, then lift both nodes simultaneously whenever up[u][k] != up[v][k]. Return up[u][0].
- **Path Distance:** dist(u, v) = depth[u] + depth[v] - 2 * depth[lca(u, v)].

B. Binary Lifting Implementation Template (C++20)

```
const int MAXN = 200005, LOGN = 20;
vector<int> adj[MAXN];
int up[MAXN][LOGN], depth_node[MAXN];

void dfs_lca(int u, int p, int d) {
    depth_node[u] = d; up[u][0] = p;
    for (int k = 1; k < LOGN; ++k) up[u][k] = up[up[u][k - 1]][k - 1];
    for (int v : adj[u]) if (v != p) dfs_lca(v, u, d + 1);
}
int get_lca(int u, int v) {
    if (depth_node[u] < depth_node[v]) swap(u, v);
    for (int k = LOGN - 1; k >= 0; --k)
        if (depth_node[u] - (1 << k) >= depth_node[v]) u = up[u][k];
    if (u == v) return u;
    for (int k = LOGN - 1; k >= 0; --k)
        if (up[u][k] != up[v][k]) { u = up[u][k]; v = up[v][k]; }
    return up[u][0];
}
```

C. Curated Practice Ladder

Problem & Platform	Tier	Core Concept / Key Intuition
CSES 1687 - Company Queries I	Core	K-th ancestor query in O(log N) using binary lifting.
CSES 1688 - Company Queries II	Core	Standard LCA template verification.
CSES 1135 - Distance Queries	Core	Compute path distance using depths and LCA.
SPOJ DISQUERY - Distance Query	Med	Extend binary lifting to store min and max edge weights on paths.

CF 208E - Blood Cousins	Med-Hard	LCA K-th ancestor + binary search on depth lists or Small-to-Large.
CF 519E - Lecture Rooms	Med-Hard	Find path midpoint via binary lifting + subtree sizes.
ICPC Dhaka Prelim 2025 - C	Med	Least Compatible Ancestor in official Dhaka Prelim contest.

4. Pillar 3: Euler Tour Technique (Flattening Trees)

A. Theoretical Foundation

- **Subtree Flattening:** Subtree of u is mapped to continuous 1D interval [tin[u], tout[u]]. Subtree updates reduce to range updates on Fenwick or Segment Tree.
- **O(1) Ancestor Check:** u is ancestor of v if and only if tin[u] <= tin[v] and tout[u] >= tout[v].
- **Path to Root:** Put +w at tin[u] and -w at tout[u] + 1. Prefix sum up to tin[u] gives path sum from root to u.

B. Euler Tour Implementation Template (C++20)

```
int tin[MAXN], tout[MAXN], timer = 0;
void dfs_euler(int u, int p) {
    tin[u] = ++timer;
    for (int v : adj[u]) if (v != p) dfs_euler(v, u);
    tout[u] = timer;
}
bool is_ancestor(int u, int v) {
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}
```

C. Curated Practice Ladder

Problem & Platform	Tier	Core Concept / Key Intuition
CSES 1137 - Subtree Queries	Core	Point update, subtree range sum query via Fenwick / Segment Tree.
CSES 1138 - Path Queries	Med	Root-to-node path queries using signed entry/exit weights.
CF 620E - New Year Tree	Med-Hard	Flatten tree + Lazy Segment Tree with 60-bit color masks.
CF 383C - Propagating tree	Med-Hard	Alternating depth updates split into even/odd Fenwick trees.
IUPC 2023 CoU - A. Ghost	Med	Tree ancestor & reachability checks via Euler Tour entry/exit times.

5. Pillar 4: Centroid Decomposition (Divide & Conquer on Trees)

A. Theoretical Foundation

- **Centroid Definition:** A node C whose removal splits the tree into components of size <= N/2. The Centroid Tree has height <= floor(log2 N) + 1.
- **Path Divide & Conquer:** Any tree path either passes through Centroid C, or lies entirely within one component. Solve paths through C in O(size), then recurse.

B. Centroid Decomposition Implementation Template (C++20)

```
bool removed[MAXN]; int sz[MAXN];
int get_sz(int u, int p) {
    sz[u] = 1;
    for (int v : adj[u]) if (v != p && !removed[v]) sz[u] += get_sz(v, u);
    return sz[u];
}
int get_centroid(int u, int p, int total) {
    for (int v : adj[u])
        if (v != p && !removed[v] && sz[v] > total / 2) return get_centroid(v, u, total);
    return u;
}
void decompose(int u) {
    int c = get_centroid(u, 0, get_sz(u, 0));
    // Process all paths passing through centroid 'c'
    removed[c] = true;
    for (int v : adj[c]) if (!removed[v]) decompose(v);
}
```

C. Curated Practice Ladder

Problem & Platform	Tier	Core Concept / Key Intuition
CF 321C - Ciel the Commander	Core	Construct centroid tree; assign ranks 'A'..'Z' to centroid levels.
SPOJ QTREE5 - Query tree V	Med-Hard	Toggle node colors; query nearest white node via centroid ancestors.
CF 161D - Distance in Tree	Med-Hard	Count pairs of nodes with distance exactly K via centroid conquer.
CF 293E - Close Vertices	Hard	Paths with length <= L and weight <= W via Centroid + Fenwick Tree.
IUPC 2023 SUST - C. Holy Tree	Hard	Centroid decomposition with sacred node distance queries.

6. Recommended 3-Week Study Schedule

Week	Topic Focus	Target Problems	Key Mastery Goal
Week 1	Diameter & LCA	CSES 1131, SPOJ PT07Z, CF 1404B, CSES 1687, CSES 1688, CSES 1135	Code binary lifting in < 10 mins; compute diameter in < 5 mins.
Week 2	Euler Tour	CSES 1137, CSES 1138, CF 620E, CF 383C, SPOJ DISQUERY, CF 208E	Flatten trees; combine Euler Tour with Fenwick & Lazy Segment Trees.
Week 3	Centroid Decomp	CF 321C, SPOJ QTREE5, CF 161D, CF 293E, IUPC 2023	Master centroid tree construction; solve path

	SUST C	counting in O(N log N).